

沈阳建筑大学本科毕业设计

机器人路径规划系统的设计与实现

Design and Implementation of Robotics Path Planning System

学 院： 计算机科学与工程学院

专 业： 计算机 2101 班

学 生 姓 名： 王文昊

学 号： 2106410104

指 导 教 师： 赵明 副教授

评 阅 教 师： 郭耸 副教授

完 成 日 期： 2025 年 6 月 6 日

沈阳建筑大学

Shenyang Jianzhu University

原创性声明

本人郑重声明：本人所呈交的毕业设计，是在指导老师的指导下独立进行研究所取得的成果。毕业设计中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究成果做出重要贡献的个人和集体，均已在文中以明确方式标明。

本声明的法律责任由本人承担。

作者签名：王文昊

日期：2025 年 6 月 6 日

关于使用授权的声明

本人在指导老师指导下所完成的毕业设计及相关资料（包括图纸、试验记录、原始数据、实物照片、图片、录音带、设计手稿等），知识产权归属沈阳建筑大学。本人完全了解沈阳建筑大学有关保存、使用毕业设计的规定，本人授权沈阳建筑大学可以将本毕业设计的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本毕业设计。如果发表相关成果，一定征得指导教师同意，且第一署名单位为沈阳建筑大学。本人离校后使用该毕业设计或与该设计直接相关的学术论文或成果时，第一署名单位仍然为沈阳建筑大学。

论文作者签名：王文昊

日期：2025年6月6日

指导老师签名：赵明

日期：2025年6月6日

摘 要

路径规划作为机器人领域的重要技术，可以用于多机器人协同作业以及在复杂环境中躲避障碍物。其中，怎样对多机器人进行路径规划以实现最优调度，怎样控制机器人的转向、前进速度，怎样实现无碰的路径规划，这些都是十分有挑战有价值的算法难题。

此机器人路径规划系统以 2023 年华为软件精英挑战赛为背景，根据赛题的需求设计了改进的 A* 路径规划算法。该算法与传统 A* 路径规划算法的启发函数不同，在启发函数中加入了近墙惩罚和经过惩罚，用以优化多机器人的路径长度与避障功能。在所设计算法的基础上，通过开发，运用 Unity 引擎通过 C# 语言实现实时物理运动与图形用户界面，使用 C++ 语言进行路径规划算法的开发。最终，为了方便用户操作形成一个完整的路径规划系统。

该路径规划系统能够生成障碍物地图，在地图上实时生成复数机器人并进行路径规划方案。在路径规划完毕后，会实时计算并把路径拆分成子节点，之后运动至目标节点。本设计的机器人路径规划模块最初存在两种缺陷：机器人在实时运动的过程中会出现转向与速度不匹配的问题而导致不规则的运动模式；运动过程中没有排除体积因素导致进入“陷阱”使机器人卡死。在对此加以优化改进后，形成了现在的路径规划系统。

关键词：机器人路径规划；A*算法；Unity 引擎

Abstract

As an important technology in the field of robotics, path planning can be used for multi-robot collaborative operation and avoiding obstacles in complex environments. Among them, how to carry out path planning for multiple robots to achieve optimal scheduling, how to control the steering and forward speed of robots, and how to realize non-collision path planning are all very challenging and valuable algorithm problems.

Based on the background of the 2023 Huawei Code Craft, the robotics path planning system is designed with an improved A* algorithm based on the requirements of the competition. Different from the heuristic function of the traditional A* algorithm, the proposed algorithm adds near-wall penalty and passing penalty to the heuristic function to optimize the path length and obstacle avoidance function of multiple robots. On the basis of the designed algorithm, through development, the Unity engine is used to realize real-time physical motion and graphical user interface through the C# language, and the path planning algorithm is developed by using the C++ language. Finally, a complete path planning system is formed for the convenience of user operation.

The path planning system can generate obstacle maps, generate multiple robots on the map in real time, and carry out path planning schemes. After the path is planned, the path is calculated and split into sub-nodes in real time, and then moved to the target node. The robotics path planning module of this design initially has two defects: the problem of mismatch between steering and speed in the process of real-time movement of the robot leads to irregular motion mode; The volume factor was not excluded during the movement, which led to entering the "trap" and causing the robot to get stuck. After optimizing and improving this, the current path planning system has been formed.

Keywords: robotics path planning; A* algorithm; Unity engine

目 录

摘 要	I
Abstract	II
引 言	1
第 1 章 系统需求分析	2
1.1 功能性需求分析	2
1.1.1 华为云机器人调度路径规划	2
1.1.2 自定义位置的寻路机器人路径规划	3
1.2 非功能性需求分析	3
1.2.1 兼容性	3
1.2.2 多进程、多线程与协程	3
第 2 章 系统设计与关键技术	5
2.1 架构设计	5
2.1.1 数据层	5
2.1.2 算法层	6
2.1.3 交互层	6
2.2 核心算法设计	6
2.2.1 经典路径规划算法介绍	6
2.2.2 A*路径规划算法	7
2.2.3 启发函数的改进	9
2.3 开发与平台	12
2.3.1 Tuanjie Engine 1.4.4	12
2.3.2 Visual Studio 2022	12
2.3.3 Red Panda Dev-C++	12
第 3 章 系统实现与优化	13
3.1 二维地图生成模块	13
3.1.1 基于 Unity 的实时地图编辑器	13
3.1.2 基于文本的华为云地图生成模块	14
3.2 路径规划的实现	15
3.2.1 路径规划的目标与分段	15
3.2.2 区分实体化障碍物	16
3.3 路径规划系统的多进程与协程	17

3.3.1 进程间的通信	17
3.3.2 Unity 协程的应用	19
3.4 实时运动	20
3.4.1 解决“抖动”	20
3.4.2 排除“陷阱”	22
3.5 图形用户界面	23
3.5.1 用户界面结构	23
3.5.2 用户界面的单例模式	24
3.5.3 图形用户界面展示	24
第 4 章 系统测试	30
4.1 测试目的	30
4.2 测试方案	30
4.2.1 硬件环境配置	30
4.2.2 白盒测试	30
4.2.3 黑盒测试	31
4.3 测试结果	32
第 5 章 技术经济可行性分析	35
5.1 复杂工程问题研究	35
5.2 技术可行性分析	35
5.2.1 现有技术支撑	35
5.2.2 跨平台可移植性	36
5.3 经济可行性分析	36
5.3.1 引擎的收费模式	36
5.3.2 部署代价	37
第 6 章 设计总结	38
参 考 文 献	39
致 谢	40
附录一 外文原文	
附录二 中文译文	
附录三 设计代码	

引 言

路径规划是在现实生活中广泛应用的技术，是运动规划的主要研究内容之一。运动规划由路径规划和轨迹运动组成，连接起点位置和终点位置的序列点或曲线称之为路径，构成路径的策略称之为路径规划。此技术在导航、城市路网规划、物流管理中均有重要作用，而把这项技术应用于机器人行业的模式即为机器人路径规划。单纯的机器人实体在没有程序与算法的协助下无法执行复杂操作，遭遇障碍时易陷入停滞，这时候机器人路径规划显得十分重要。

机器人路径规划核心在于对多目标、多变量以及多约束的模型进行优化求解。对于非完整约束的机器人来说在包含障碍物的环境中求解最优路径是 NP-hard 问题，即对于任意场景无法保证在多项式时间内求得最优解。所以当今大部分规划算法追求的是次优解或局部最优^[1]。

许多的路径规划一般专注于路径的静态规划，而本设计的重点和解决思路不仅仅聚焦于路径规划，更多的是讨论在多机器人并发的场景下如何有效的规划机器人以及避免碰撞，同时对于在实际物理引擎环境下模拟运动过程中可能产生的问题并进行解决。

本毕业设计起源于 2023 年华为软件精英挑战赛，基于赛题开发了基础的交互程序，通过输出交互指引机器人路径规划。所以选择《机器人路径规划系统的设计与实现》为毕业设计题目一方面是因为此设计与 2023 年参与的华为软件精英挑战赛赛题具有部分一致性，另一方面也是为了完善以前的程序。得益于 Unity 引擎优秀的物理模拟效果，开发出了实现实时运动的路径规划模块，并将前后开发的模块通过进程间的通信整合成一个完整的系统。使用 Unity 的画布功能实现了图形用户界面，基于设计的三层架构分块设计界面于按钮，并把模块功能于图形界面联系起来，使得操作更加便捷而非使用指令调用。而在物理实现上，初期开发过程中发现实时运动会出现转向与速度不匹配的问题而导致“抖动”运动，这是由于 4 个方向的搜索无法考虑斜向运动而产生的；也发现运动过程中若没有排除体积因素可能会使机器人进入“陷阱”而卡死，通过检索地图可以解决这一问题。最后在经过一系列的测试和评估下，此机器人路径规划系统的设计与实现才得以完成。

第 1 章 系统需求分析

1.1 功能性需求分析

1.1.1 华为云机器人调度路径规划

机器人已经在智能制造、物流仓储、递送、医疗等很多领域显现出巨大的市场需求与商业价值，在此背景下华为在 2023 年举办的华为软件精英挑战赛就以此为赛题（华为云数据中心运维机器人如图 1.1 所示）。



图 1.1 华为云数据库中心运维机器人

在机器人领域，规划多机器人的路径加上调度算法，控制机器人转向与前进速度以实现无碰的最优路径等都是非常有价值的算法难题。2023 年的华为软件精英挑战赛上华为官方通过软件模拟了多机器人的运行环境以及真实机器人的状态信息，由选手来挑战这些有价值的算法难题。

为了满足这类有实际任务的机器人路径规划需求，此次设计以 2023 年参与的华为软件精英挑战赛实际程序以及华为官方判题器为基础。其中挑战赛要求开发者设计一个交互程序，通过输出方式来与判题器交互，指定四个机器人前往复数的工作台实现各类操作。而本设计通过改进 A* 路径规划算法、运用管道通信来与判题器交互数据，实现了机器人避障寻路满足了赛题需求。之后使用进程通信将其整合进系统中直接使用图形用户界面调用。

1.1.2 自定义位置的寻路机器人路径规划

在无特殊任务要求的路径规划需求中，算法生成的目的地序列不一定是用户觉得合适的结果。区别于上文有实际任务的机器人路径规划需求，在机器人没有明确的任务节点时，仅仅只靠算法决定目的地，将无法满足用户的主观意愿，此时路径规划算法所认为的最优将不再是最优。例如，导航系统的最短路径不会考虑用户是否有风景需求与购物需求而规划路径。且机器人路径规划中用户可能会在某些情况下无视障碍使机器人不避障、用户也可能需要机器人按照自己指定的节点移动而非程序规划的路径节点移动。所以制作能自定义位置的机器人路径规划模块能方便用户设置，满足用户需求。

本设计使用 Unity 引擎开发了能够自定义位置的机器人路径规划功能。使用者能够根据自己的操作指定机器人到达特定的位置，能自定义机器人的数量以及编辑障碍物，以满足使用者的自己自定义路径节点的需要。

1.2 非功能性需求分析

1.2.1 兼容性

用户的设备形形色色，十分多样化。因此当设计一个系统时应为不同设备、不同平台开发对应版本。但是大量的重新开成本过大，因此如何在小规模改动甚至无需改动的情况下实现不同平台的稳定运行是一个十分重要的需求。

Unity 引擎的打包功能使开发出的项目在兼容性方面表现出色，其开发出的项目可被打包到多种平台上运行，同时也可适配不同类型的设备，这一功能打破了因跨平台而需对整个项目进行重构的困境，节省了开发成本，还契合了不同环境下用户的需求，使项目更具传播性与实用性。

Unity 引擎当前已支持 Windows、Mac、Linux 操作系统、Android、iOS、WebGL、Mini Game、HMI Android、QNX、Embedded Linux 平台，OpenHarmony 开源项目等。

1.2.2 多进程、多线程与协程

由于本设计基于 2023 年华为软件精英挑战赛开发了路径规划模块，而后又结合 Unity 引擎重新设计了实时运动模块和用户界面模块。为了能够整合这两个时间段开发的模块，需要使用到进程间的管道通信。而 Unity 路径规划模块中需要使用协程来确保主线程的流畅，因此此部分也是一个十分重要的需求。

进程作为操作系统实施资源分配的基础单元，呈现着程序动态执行的具体示例。每一个进程都有独立的内存空间，涉及代码段、数据段以及堆栈等部分，操作系统借助进

程控制块来管理其状态、权限以及资源分配。进程和线程共享内存的情况不一样，进程之间有着严格的隔离，需要依靠进程间通信机制，像是管道、共享内存、套接字等来达成数据交互。

线程是操作系统任务调度的最小执行单元，隶属于进程并共享其内存资源。与进程独立内存空间不同，同一进程内的线程可并发执行代码段，通过共享内存实现高效数据交互提升程序执行效率。多线程技术通过时间片轮转或并行处理机制，使单核 CPU 实现伪并行，多核 CPU 能真正同步执行任务。

协程作为一种协作式程序组件，它的核心机制是借助相互依赖的协程序列来实现 CPU 资源的轮流占用，在任何时刻，都只允许单个协程处于运行状态，其余的协程则处于休眠状态，这种用户态轻量级线程拥有独立的寄存器上下文与栈结构，它的一个关键特征是在执行过程中可主动暂停，并完整地保存运行状态。当恢复执行时，可以精确地回溯到中断位置继续执行，与系统调度的线程机制不同，协程是依靠程序自主控制来实现执行权的让渡，将 CPU 资源转移到指定的协程，而不是被动地接受内核调度中断，这种在用户空间内的自主切换机制避免了内核介入所带来的性能损耗，使得上下文切换的开销降低，在资源占用方面相比传统线程有一定优势^[2]。

(1) 线程（Thread）

定义：线程是进程中的一个执行任务，是处理器任务调度和执行的基本单位。

应用：多用于并发量大的场景。线程的切换发生于用户态，系统开销小。

(2) 进程（Process）

定义：进程是操作系统资源分配的基本单位。它是程序的一次执行过程，拥有独立的内存空间。

应用：调用外部.exe 程序。进程的切换发生于内核态，系统开销大。

(3) 协程（Coroutine）

定义：轻量级协作式任务，由程序自行调度，无并行执行能力。

Unity 应用：实现主线程分时异步操作，避免阻塞但非真正多线程。

综上所述可以如表 1.1 所示：

表 1.1 多线程、多进程与协程的对比

特性	多线程	多进程	协程
内存空间	共享进程空间	完全独立	独立
资源开销	中	高	极低
执行方式	并行/伪并行	独立并行	主线程内消息传递

第 2 章 系统设计与关键技术

2.1 架构设计

设计架构类似于一个高楼的地基与框架，没有浇筑与规划好地基与框架就无法铸成高楼。机器人路径规划系统的架构具体分为三层，分别为数据层、算法层与交互层。具体架构如图 2.1 所示：

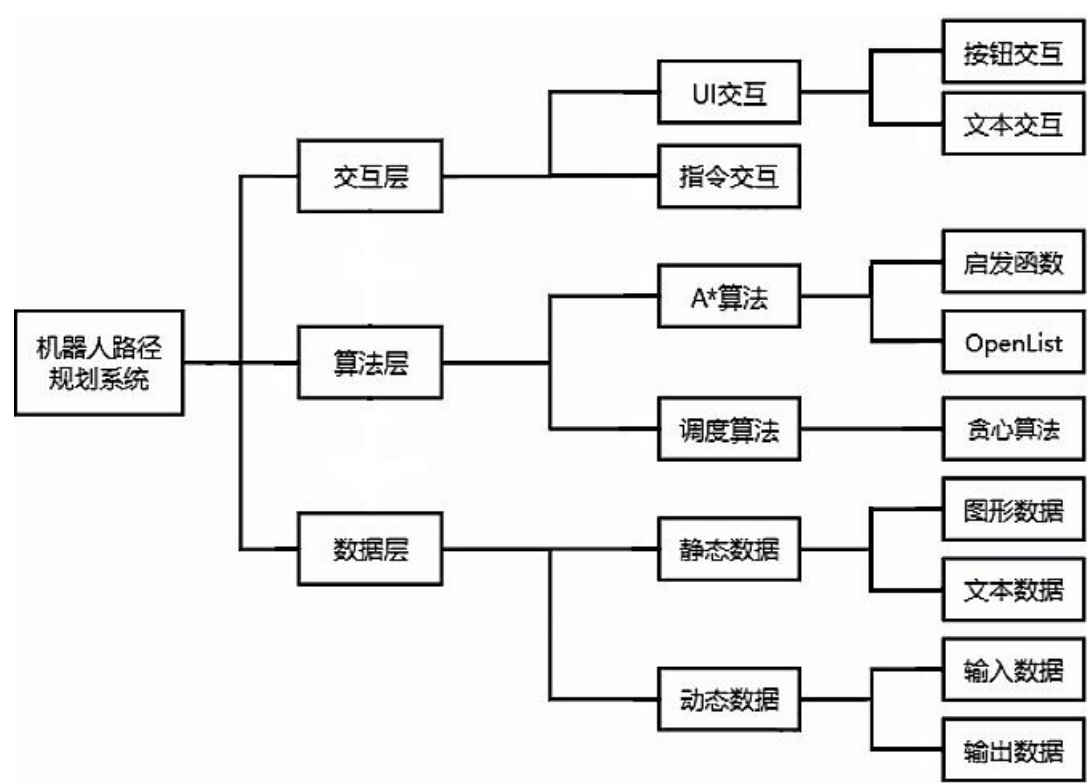


图 2.1 机器人路径规划系统的架构设计

2.1.1 数据层

现如今任何算法与程序都离不开数据的支持，没有数据的程序是没有灵魂的。本设计的数据分为静态数据与动态传输数据：

- (1) 在静态数据上，主要用于保存固有信息。用于基于文本的地图生成模块和 Unity 引擎内预制体（prefab）和场景、图像、文字等组件的数据保存。
- (2) 在动态传输数据上，由于需要进程间相互通信。即进程间的管道通信用来在 Unity 主进程和判题器的交互以及判题器和 C++的源文件交互数据。

2.1.2 算法层

算法作为计算机科学的核心，几乎贯穿了计算机技术的所有领域。当今信息化的时代，算法已经深入到计算机的方方面面。

本设计的核心算法是 A* 路径规划算法，以该算法为核心规划多机器人的路径。同时为了满足华为软件精英挑战赛这类拥有实际任务的机器人路径规划需求，也加入了调度算法。具体实现可见 A* 路径规划算法与启发函数的改进。

2.1.3 交互层

交互层作为软件架构中的顶层模块，承担着用户与系统间的信息交换职责，是整个架构的枢纽，其设计直接影响用户体验与系统可用性。

本设计通过使用 Unity 引擎的 UI 技术在画布（Canvas）上设计按钮与文本框，以及可以输入指令来执行华为云判题器和 C++ 源文件。具体实现可见用户图形界面的实现。

2.2 核心算法设计

2.2.1 经典路径规划算法介绍

常见的路径规划算法有 Dijkstra 算法、A* 算法、D* 算法、Floyed 算法。还有一些在这些算法上进行拓展的算法如^[3]中的 JSP 挑点搜索算法在 Unity 项目开发中也广泛应用。

Floyed 算法是多源最短路径算法，用来求解图中任意两节点间的最短路径。一开始会构建一个 $n \times n$ 的距离二维矩阵 $D[n][n]$ ，其中 $D[i][j]$ 表示节点 i 到节点 j 的直接距离，若两节点无直接连接则设为无穷大 ∞ （实际程序实现时为 0x3f3f3f3f 以保证此“无穷大”在松弛操作后加上一个数不会变为负数，即防止溢出）。之后遍历所有可能的中间节点 k （从 1 到 n ），并对每对节点从 i 到 j 检查是否存在更优路径：

$$D[i][k] + D[k][j] \quad (2.1)$$

若存在，则更新 $D[i][j]$ 为更小的值，数学表达为：

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j]) \quad (2.2)$$

Floyed 算法对存在负权边的图提供支持，不过图里不能有负权回路，不然路径长度会一直减小，它的时间复杂度是 $O(n^3)$ ，空间复杂度是 $O(n^2)$ ，适用于像物流网络这类中小规模图的全局路径规划，比如在物流配送问题里，Floyed 算法能预先算出所有仓库和客户点之间的最短路径，优化配送路线并降低成本。然而立方级的时间复杂度 $O(n^3)$ 让它难以处理超大规模图数据，这时要结合分治策略等方法来优化。

Dijkstra 算法是一种经典的图搜索算法，该算法的空间复杂度是 $O(V)$ ，时间复杂度可大致为 $O(n^2)$ ，具体取决于所使用的数据结构和图的特性。Dijkstra 算法专注于求解单一起点到图中所有其他节点的最短路径。采用贪心策略，通过逐步扩展当前已知的最短路径来探索网络，其核心在于维护一个优先队列，每次从中选取距离起点最近的未处理节点作为跳板，更新其相邻节点的路径权重。这种“由近及远”的探索方式确保了首次到达目标节点时获得的路径必然最优。该算法在静态环境中的物流配送、网络路由规划等场景表现突出，例如仓库机器人可借助其计算前往货架的最短移动路线。然而，由于需要遍历所有可能节点，其计算效率随网络规模扩大而显著下降，且无法处理负权边情况，这些局限促使了后续 A* 路径规划算法等启发式算法的演进。

A* 路径规划算法作为经典的启发式搜索算法，在全局路径规划算法里有着快速、高效且准确的特性。故而在智能驾驶、机器人路径规划等问题方面得到了广泛应用[1]。A* 路径规划算法借助融合实际路径代价和启发式预估代价，于路径规划中达成效率与精度的平衡。该算法把起点当作初始节点逐步拓展相邻网格并计算代价值，优先去拓展启发函数更小，即代价更小的区域。在仓储机器人调度过程中，A* 路径规划算法凭借动态调整节点权重，可有效避开密集障碍区域，同时还可以缩短全局路径搜索时间。和传统广度优先策略相比，其搜索范围明显缩小，适用于复杂环境下的多机器人目标协同规划。

在此机器人路径规划系统的设计中，选择算法要综合考量诸多需求。由于此设计采用 A* 路径规划算法能够实现多机器人的并发路径规划且互不干扰，在避障寻路上更具独立性。同时 A* 路径规划算法能够通过不同的权值加减来影响机器人路径，在避障寻路上更具目的性。综上考虑选择 A* 路径规划算法是

2.2.2 A* 路径规划算法

A* 路径规划算法执行过程开始于开放列表（OpenList）与关闭列表(CloseList)的初始化，起点被给予实际当前代价与估计代价。在迭代过程中，优先挑选代价值最低的节点进行扩展，要是抵达终点就反向追溯完整路径，针对相邻节点的处理包括障碍检测与代价值更新，以此保证路径可行性[4,5,6,7,8,9]。

A* 路径规划算法作为一种被广泛应用的启发式广度优先搜索算法，其具体算法流程图如下一页的图 2.2 所示。

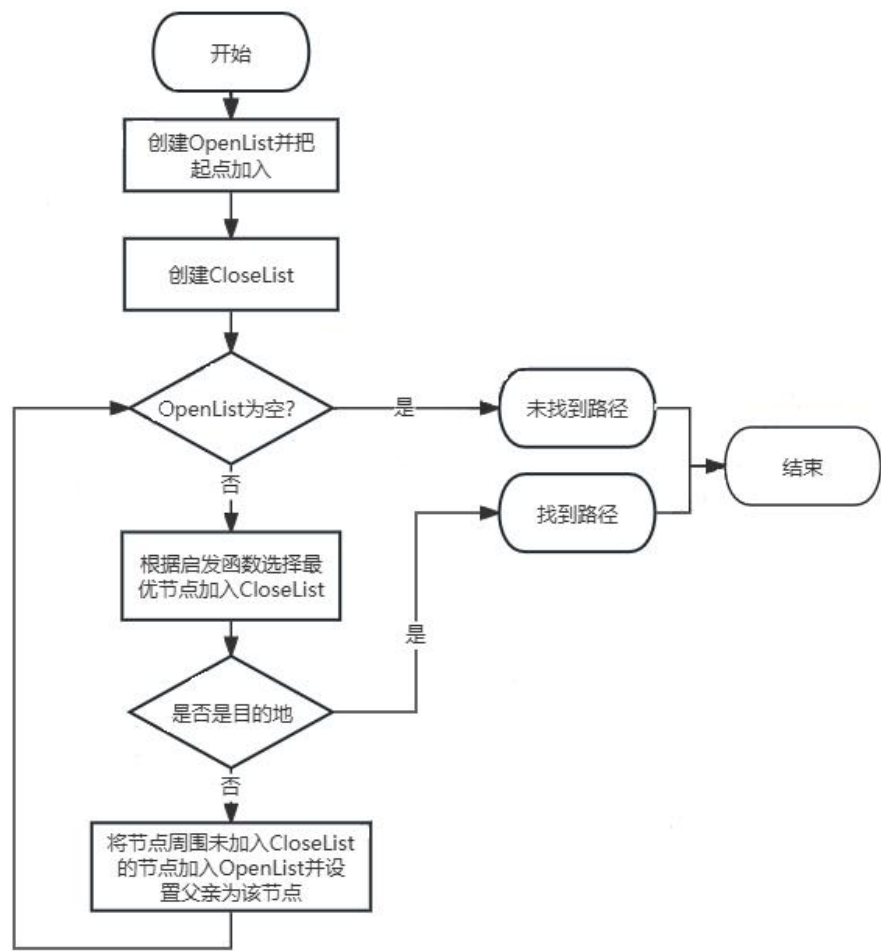


图 2.2 A*路径规划算法流程图

传统的 A* 路径规划算法在寻找子节点的过程中，需要对当前节点周围的 8 个方向进行遍历。在搜索路径的过程中，首先遍历所有 OpenList 中的元素后，选择出启发式函数值最小的节点，并将该节点放入 CloseList 中。然后将当前节点周围的 8 个节点放入 OpenList，重复此过程直到找到终点。已加入 CloseList 的节点将不再加入 OpenList，即相当于一种标记防止重复搜索。流程图如下图 2.3 所示：

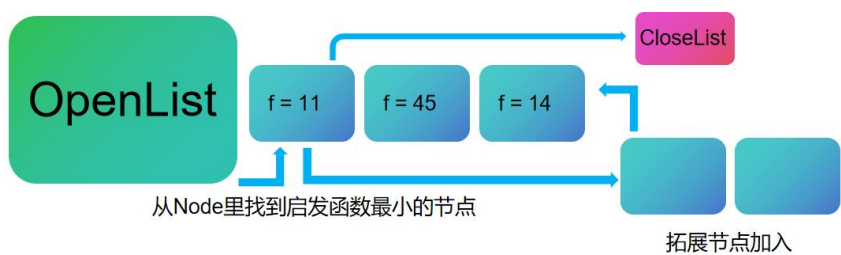


图 2.3 OpenList 与 CloseList

2.2.3 启发函数的改进

启发函数的主要作用是把无目的地搜索转变为有指向性地搜索，它通过估计代价来决定下一步的最佳路径。在 A* 路径规划算法中，启发函数被用于计算每个节点的评价函数，该函数由两部分组成： $g(n)$ 和 $h(n)$ 。其中， $g(n)$ 是从起始节点到当前节点的实际代价，而 $h(n)$ 是从当前节点到目标节点的估计代价。

A* 路径规划算法的启发函数基础为：

$$f(n) = g(n) + h(n) \quad (2.3)$$

本设计中 $h(n)$ 为当前节点到目标节点的曼哈顿距离，为上一节点 G 值+两节点之间的移动代价。而本设计基于基础的启发函数加入了 $s(n)$ 与 $c(n)$ ，即：

$$f(n) = g(n) + h(n) + s(n) + c(n) \quad (2.4)$$

$s(n)$ 为近墙惩罚。 $c(n)$ 为有机器人或将有机器人经过的惩罚。

实际代价 $g(n)$ ：实际代价即为从初始节点到达此节点的路径代价。本设计对于实际代价规定水平移动的代价以 10 为基。所以可以求得斜向移动的代价为：

$$\sqrt{10^2 + 10^2} \approx 14 \quad (2.5)$$

从初始节点来看，节点的拓展实际代价如图 2.4 所示。之后再拓展结点的时候会初步累加实际代价。

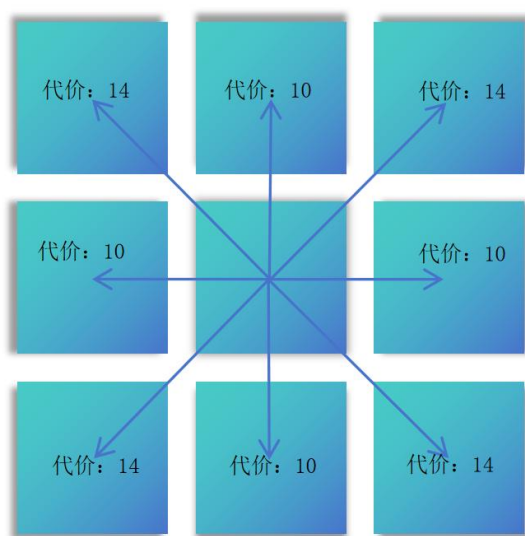


图 2.4 实际代价的处理

估计代价 $h(n)$:本设计的估计代价采用曼哈顿距离,相当于只考虑当前节点到目标节点的 X 轴水平距离和 Y 轴垂直距离。估计代价不考虑其他的任何因素,仅仅只估计此节点到目标节点的大概距离。曼哈顿距离计算公式如下,其中 $h(n)$ 为曼哈顿距离, $x1$ 与 $y1,x2$ 与 $y2$ 分别为起点与目标点各自的坐标值:

$$h(n) = |x1 - x2| + |y1 - y2| \quad (2.6)$$

近墙惩罚 $s(n)$:近墙惩罚值为100,实际代码如下:

```
if(if_near_wall(&this_node)) array_node[this_node.x][this_node.y].S = 100;
```

若当前节点通过`if_near_wall`函数判断后返回`true`,则当前节点的 S 值将从初始值的0设置为100。之后在加入`OpenList`时会将启发函数 $f(n)$ 求出,此时相对于没有 $s(n)$ 的标准 A^* 路径规划算法而言,此设计的机器人会偏向于不靠近墙壁移动。因为考虑到物理引擎中可以模拟实际机器人的体积碰撞,所以近墙惩罚是必须的。如下图2.5所示:

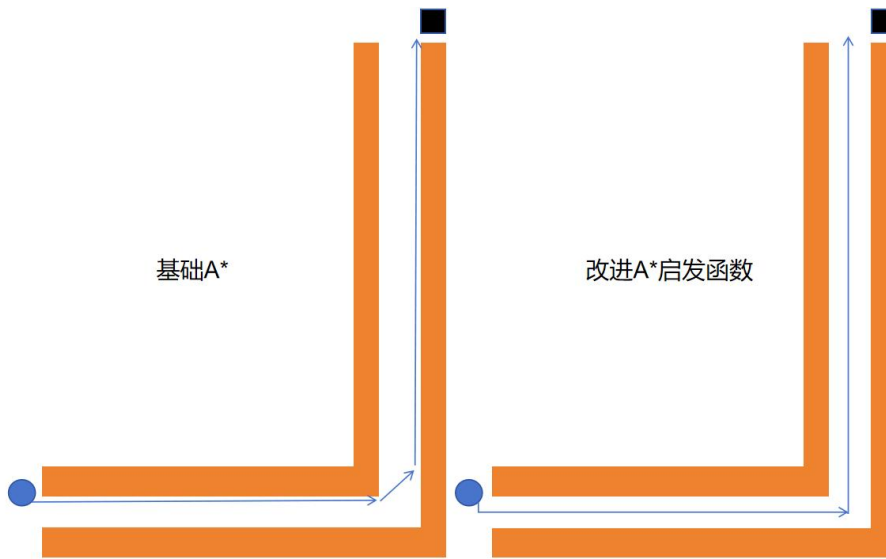


图 2.5 加入近墙惩罚后的路径规划

经过惩罚 $c(n)$:由于本设计不仅仅局限于单机器人的单目标规划,所以在多机器人的调度下难免会发生路径占用导致的碰撞。于是给启发函数加上经过惩罚代码模块如下:

```
if(vis[this_node.x][this_node.y]!='.'){//字符'.'表示没有被占用。
```

```
array_node[this_node.x][this_node.y].C = 3000;
```

```
array_node[this_node.x+1][this_node.y].C = 2000;
```

```
... ..//以下代码基本重复,是给周围节点给予不同的C值}
```

所以当某节点已被加入某机器人的路径时，其他机器人由于启发函数经过惩罚 $c(n)$ 的原因更倾向于绕道而行，除非其他路径的代价超过额外添加的 3000 惩罚值，机器人才有可能行驶在同一条道路上。如果没有加入经过惩罚，则所有机器人都会优先考虑自己的最近路径而不考虑是否会碰撞，使得机器人会拥挤于一条路口上。具体可以用图 2.6 简单表示。

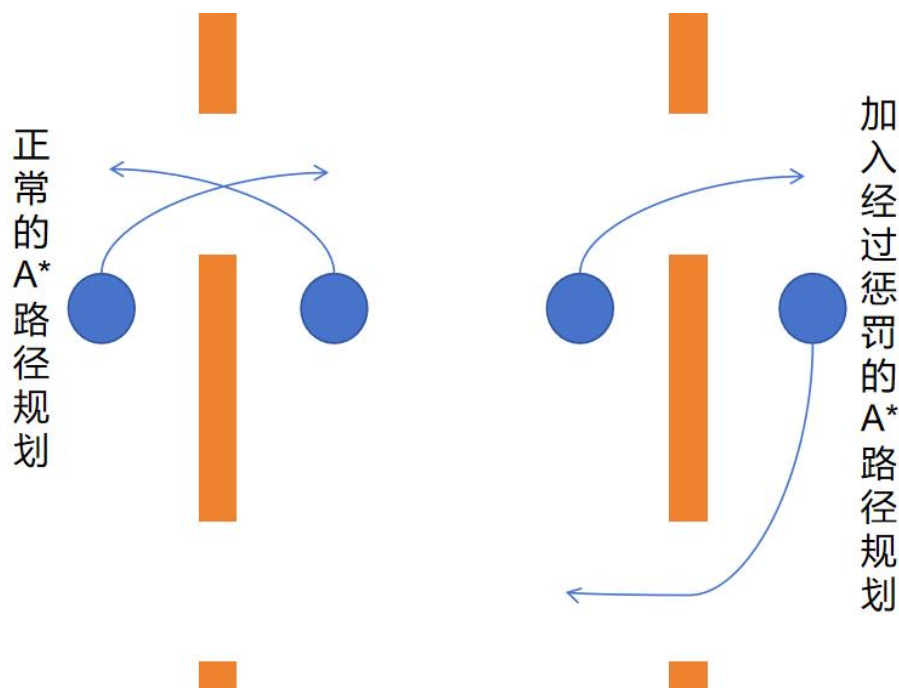


图 2.6 加入经过惩罚后的路径规划

经过以上的设计分析对 Dijkstra 算法、Floyed 算法、A*算法以及改进启发函数的 A*算法可以获得如下结论，如表 2.1 所示：

表 2.1 关于上文介绍的路径规划算法对比

	Dijkstra 算法	Floyed 算法	A*算法	本文改进的 A*算法
目标	单源最短路径	多源最短路径	单源最短路径	多源最短路径
数据结构	优先队列	邻接矩阵	优先队列	优先队列
考虑对撞	不考虑	不考虑	不考虑	考虑
启发函数	不需要	不需要	需要	需要

2.3 开发工具与平台

2.3.1 Tuanjie Engine 1.4.4

团结引擎（本设计简称 Unity 引擎）是一款专门为中国开发者打造的 3D 引擎，发布于 2024 年，是在 Unity 2022 LTS 版本基础上进行了改进的本土化版本，其设计目的就是为契合中国市场的需求，团结引擎有诸多功能优势：

(1) 将本土文化与市场特色相结合，以此契合中国开发者的独特需求，为创作给予全面的支持。

(2) 经过多项有原创性的本土优化与拓展，引擎已经可兼容适配中国科技生态内的众多软硬件平台，像目前它已支持 Windows、Mac、Linux 操作系统，以及 Android、iOS、WebGL、Mini Game、HMI Android、QNX、Embedded Linux 平台、OpenHarmony 开源项目等，并且即将支持 AliOS 操作系统。

(3) 有多平台支持能力，把小游戏与之深度优化开发体验相结合，可提供全方位的性能提高以及工具支持，这样在让开发者高效创作之时，还可以保证小游戏拥有流畅的运行和出色的体验。

(4) 界面友好且有高度可定制性的编辑器，再结合车机创新应用模板，能使开发者依据项目需求轻松调整和优化开发工具，节省时间并提升效率。

2.3.2 Visual Studio 2022

Visual Studio 2022 是由微软公司推出的集成开发环境，于 2021 年正式发布，是首个原生支持 64 位架构的 Visual Studio 版本。作为常用的软件开发工具，可支持的组件包括 Web 和云、桌面应用和移动应用、游戏、其他工具集，支持 C# 与 C++ 等各种语言。

2.3.3 Red Panda Dev-C++

Red Panda Dev-C++ 是在原版 Dev-C++ 基础上优化而来的开源集成开发环境，它在用户体验、教学支持以及现代开发需求等方面都做出了创新改进，Red Panda Dev-C++ 和原版 Dev-C++ 有所不同，Red Panda Dev-C++ 的代码补全功能更为完善，还有自动检测功能，并且拥有丰富的编程示例库，这些特性使得它对于初学者来说更容易上手。

第3章 系统实现与优化

3.1 二维地图生成模块

3.1.1 基于 Unity 的实时地图编辑器

(1) 预制体

Unity 中的预制体是一种可重复使用的游戏对象模板，可提高开发效率和资源管理能力，预制体将配置完整的游戏对象，像组件、子对象以及属性等保存为独立资源，可在场景中迅速实例化多个副本，并且可以保证所有实例和模板同步更新，保存为预制体后，只需修改模板就能对所有实例的属性进行全局调整，无需再逐个手动修改，省去了繁杂的操作。在性能优化层面，预制体依靠减少重复资源加载以及统一管理材质、碰撞体等组件，有效降低了内存占用和渲染开销，凭借合理设计预制体结构，开发者在保证功能一致性的情况下，可提升项目的可维护性与扩展性。

类似于 C++ 中的类与对象。Prefab 相当于类，而通过各种方法生成的机器人则对应实例化的各个对象，每个实例化的对象都继承 Prefab 的组件且可以修改。与 C++ 实例化对象不同的是，Prefab 可以通过 Unity 的可视化窗口拖拽与复制生成。可是这种方法效率低下，且只能由开发者用源工程文件实现。所以通过代码生成的 Instantiate 方法是处理不确定数量的实例的最佳方法，如下图 3.1 所示。

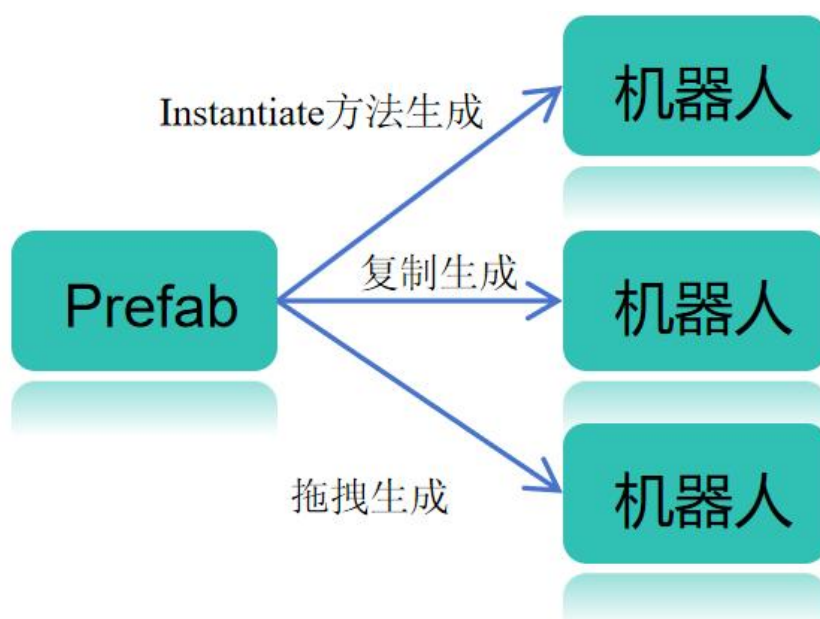


图 3.1 Prefab 生成实例对象

(2) 创建障碍物与机器人

Unity 预制体的模块化设计简化了障碍物的创建流程，可支持快速生成带有物理属性和交互逻辑的 `GameObject`，开发者可以预先配置好障碍物与机器人模型、碰撞体以及材质，将包含 `BoxCollider2D` 与 `Rigidbody2D` 组件的游戏对象保存为预制体模板。

在动态场景里，借助脚本调用 `Instantiate` 方法或者通过 Unity.UI 给本设计提供的文本输入框（Text Mesh Pro）输入具体数据的操作，实现障碍物或者机器人的程序化生成，采用对象池管理频繁生成销毁的障碍物实例来优化性能，减少运行时内存分配的开销，利用层级碰撞矩阵屏蔽非必要交互，提升物理计算效率，这种方式在迷宫导航训练系统中得到验证是有效的，兼顾了场景复杂性与实时性能要求。

3.1.2 基于文本的华为云地图生成模块

在华为云的路径规划模块中，地图由二维数组存储。其中“#”为墙壁；“.”为空地；“A”代表机器人；数字 1-9 代表工作台。例如一个地图大致可以如下图 3.2 所示：.

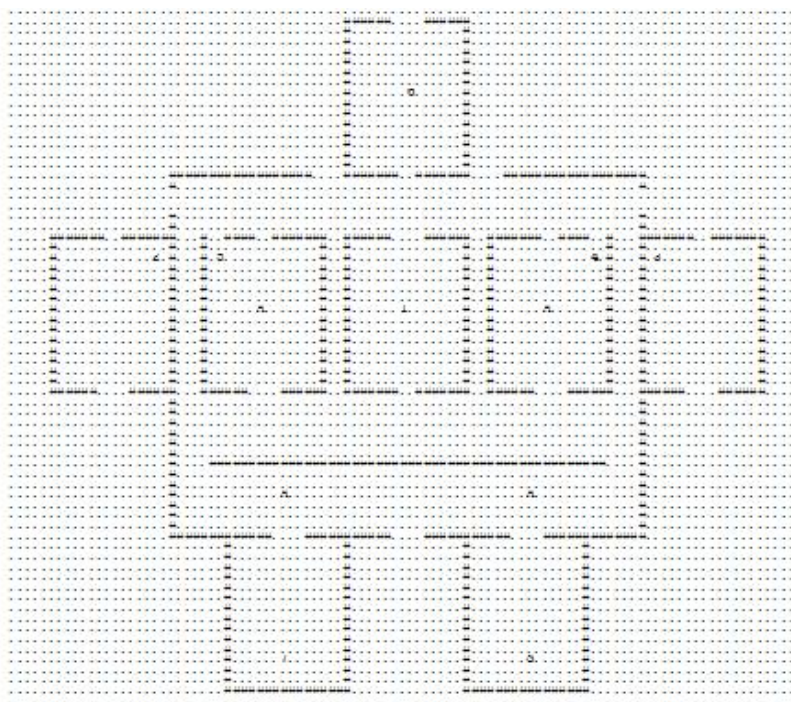


图 3.2 文本地图示例

一般来说可以通过 `cmd` 来启动 `robot_gui.exe` 判题器与 Devc++编写的代码进行 pipe（管道）通信。启动通信的外部调用方法：

`robot_gui.exe bs\5-8\change -m maps\2.txt`（-m 为地图选择，-f 为快速模式）

若想要改变地图与创建地图,只需要自己创建一个.txt 文件,输入矩阵 `map[101][101]` 大小相同的 100*100 的文本地图即可, 然后保存至 `maps` 文件夹内。倘若想要加载自己设计的地图, 可以在系统内部打开地图指令加载, 在其中输入:

`-m maps\XXXX.txt` (其中 XXXX 代表自己地图的命名)

3.2 路径规划的实现

3.2.1 路径规划的目标与分段

(1) 路径规划的分段

每一个机器人在每一次的路径规划中都只有一个终点目标,但是可以有許多个小目标节点,在到达一个小目标节点后会转而往下一个小目标节点移动。将长段路径划为多个小目标的直线路径是路径规划中最实用技巧。每一次的小目标相对与机器人来说都是一个简单的移动,无需考虑障碍物:因为算法在规划路径的过程中把路径拆分的过程已经把障碍物排除在外了,具体如图 3.3 所示。

每次规划一个机器人的路径时会调用本设计的改进 A*路径规划算法,然后根据算法返回路径,但是根据矩阵的路径确定而来的小目标会使得小目标过于繁多,浪费内存空间。因此可以判断小目标是够处于转角,若处于在转角则加入路径栈,否则直接丢弃。之后每当机器人经过一个小目标节点的同时, `Pop()`出栈以获得下一个小目标。

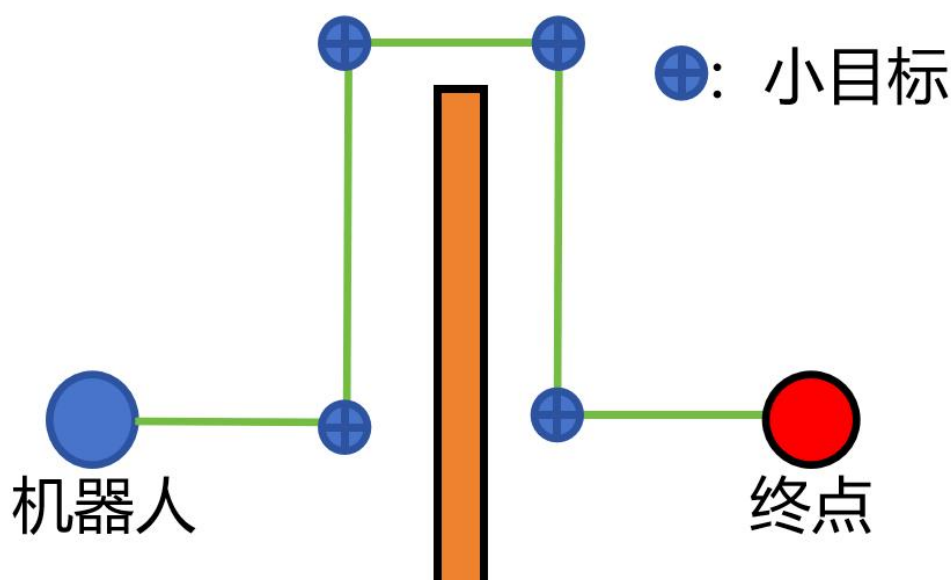


图 3.3 路径规划的分段

(2) 路径规划的目标匹配

如上文介绍的可以知道机器人会分成很多个小目标，那么当机器人与终点成对出现时，如何区分每个机器人和每个目标的情况呢？

首先是基于华为云机器人路径规划模块的处理情况：为每个机器人单独开一个栈存储其路径节点情况。这方法简单易实现，而且由于每个栈内存空间的独立，各个路径的数据不会相互干扰。

然后是基于 Unity 引擎实现的路径规划系统，其处理方法为：为每个节点确定一个标签，如 `target0`、`target1`、`target2` 此类规则命名。然后通过 `GameObject.Find("target" + num)` 函数找到对应的命名，然后再 `GetComponent<Transform>().position` 获取其二维坐标即可。

3.2.2 区分实体化障碍物

与华为云的路径规划模块不同，Unity 中的地图障碍物无法用简单的二维数组中的“#”表示。所以为了确定障碍物的大小以及机器人路径规划过程中的避障功能，本设计使用到了 Unity 引擎的 `Rigidbody2D` 和 `Collider2d` 组件。

(1) Rigidbody2D（2D 刚体）

Unity Editor 的 2D 刚体组件定义游戏对象及其子游戏对象在场景中的定位、旋转和缩放方式。更改 2D 刚体将更新其他组件，这样可以更新渲染对象的位置或碰撞体所在位置之类的属性。Unity 引擎提供的 2D 刚体能够移动碰撞体并使这些碰撞体彼此交互，因此物理模拟才得以实现。在 Unity 中刚体组件提供以下属性：

- ① 质量：控制物体的惯性，质量越大，惯性越强。
- ② 阻力：影响物体线性运动的空气阻力，数值越大，物体移动时减速越快；若设为极大值，物体会立即停止。
- ③ 角阻力：类似阻力，但作用于物体的旋转运动，阻碍其旋转速度。
- ④ 使用重力：勾选后，物体受重力影响自然下落；取消则忽略重力。

除了以上还有很多此类的接口与设定可供使用，Unity 引擎的强大物理引擎为本设计提供了完美的模拟环境，而由于这些功能的实现与本设计无关，所以不再赘述。

(2) Collider2D（2D 碰撞体）

Unity 的 `Collider2D` 组件是 Unity 物理引擎交互里的关键部分，它依靠几何形状来区分对象的碰撞边界，常见的 `BoxCollider2D`、`CircleCollider2D` 以及 `PolygonCollider2D`（如下图 3.4 所示）分别为矩形、圆形以及自定义多边形轮廓。

加入了该组件后，Collider2D 会随着 GameObject 的移动而移动，在此期间若碰到其它非 is Trigger 的 Collider2D 时，会执行碰撞判定，将重叠的 Collider2D 部分弹开并给予反向的加速度来达到碰撞的效果。



图 3.4 BoxCollider2D、CircleCollider2D 及 PolygonCollider2D

在路径规划系统里面，Collider2D 用来对障碍物作出标记，并且和 Rigidbody2D 的动力学属性相结合，以此达成机器人碰撞反馈，开发者可借助 is Trigger 属性来切换碰撞器行为，在非触发模式时，物理引擎会自行计算刚体之间的力作用，而在触发模式时，仅仅是检测重叠事件，以便用于逻辑回调，像是收集物品或者区域进入检测之类的情况。

3.3 路径规划系统的多进程与协程

3.3.1 进程间的通信

本设计借助多进程间的管道通信，为华为云机器人调度提供路径规划，管道通信作为本次设计进程间通信核心机制之一，借助内核维护的缓冲区达成单向数据流传输，于系统级任务协同和数据交换方面拥有独特优势，管道本质上可视为虚拟文件系统里的特殊文件，由操作系统内核分配固定大小的环形缓冲区。

本设计的进程通信结构可以看成生产者-消费者模型，bs\5-8\change 是生产者，负责生产每帧通信需要的数据块。robot_gui.exe 是消费者，在设计里负责接受管道内的数据块。Windows 平台借助 Create Pipe() API 实现类似功能，不过其底层采用命名管道机制，支持跨网络通信，和共享内存或消息队列相比，管道通信有严格的时序性，即数据读取顺序与写入顺序严格一致，且遵循“先进先出”原则。具体图例如图 3.5 所示。

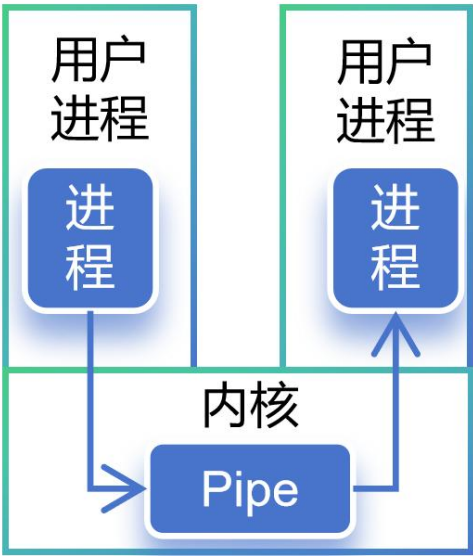


图 3.5 Pipe 管道通信

以下是关于 robot_gui.exe 和 bs\5-8\change 的通信指令：

初始化：读入地图完成初始化后程序应输出 OK 告诉 robot_gui.exe 已就绪。

每一帧交互：

- 第一行，程序需要输出一个整数，表示帧 ID。
- 紧接着，每行一个指令，按照"指令<机器人 ID>[参数 2]"的格式进行输出，机器人 ID 的取值范围是[0,3]（即四个机器人），对应机器人的输入顺序。支持的指令如下

表 3.1 所示：

表 3.1 判题器的交互指令

指令	参数 1	参数 2	说明
forward	机器人 ID 取值[0, 3]	[-2,6]之间的浮点数	设置前进速度单位为米/秒。 正数前进负数后退。
rotate	机器人 ID 取值[0, 3]	[-2,6]之间的浮点数	设置旋转速度单位为弧度/秒。 负数顺时针正数逆时针旋转。
buy	机器人 ID 取值[0, 3]	无	购买当前工作台物品以输入数据的 所处工作台 ID 为准。
sell	机器人 ID 取值[0, 3]	无	出售物品给当前工作台，以输入数 据的所处工作台 ID 为准。

输出的指令将按顺序依次执行，同一个机器人可以在同一帧内执行多条指令，因此机器人可以同一帧内出售后立刻买入。当你输出完所有指令后，紧跟一行 OK，表示输出结束。例如，一个可能的输出是：

1140	//代表此时为第 1140 帧
forward 0 4.5	//0 号机器人向前行走速度为 4.5
rotate 0 3.14159	//0 号机器人旋转
sell 1	//售出物品 1
buy 1	//购买物品 1
OK	//表示一帧的指令交互完成

3.3.2 Unity 协程的应用

Unity 协程作为一种在单线程里的分时任务调度机制，在 A* 路径规划算法里借助异步化处理解决了大规模路径搜索致使的帧率卡顿问题。由于 Unity 引擎的特性,其运行时没有 main() 函数一说，取而代之的是 Update() 函数。Update() 函数的作用是每一帧调用此函数内的所有函数。而传统 A* 路径规划算法在遍历开放列表 (OpenList)、扩展相邻节点时要是一次性做完所有计算，很容易造成 Unity 主线程阻塞，在复杂网格环境或者移动设备上可能会引发界面冻结。协程借助 yield return 指令把算法拆分成多帧来执行，像单帧处理不超过 50 个节点扩展操作，剩余计算延迟到后续帧完成，以此保证每帧耗时控制在合理范围之内，维持画面渲染以及用户交互的流畅性，具体原理可见图 3.6 所示。

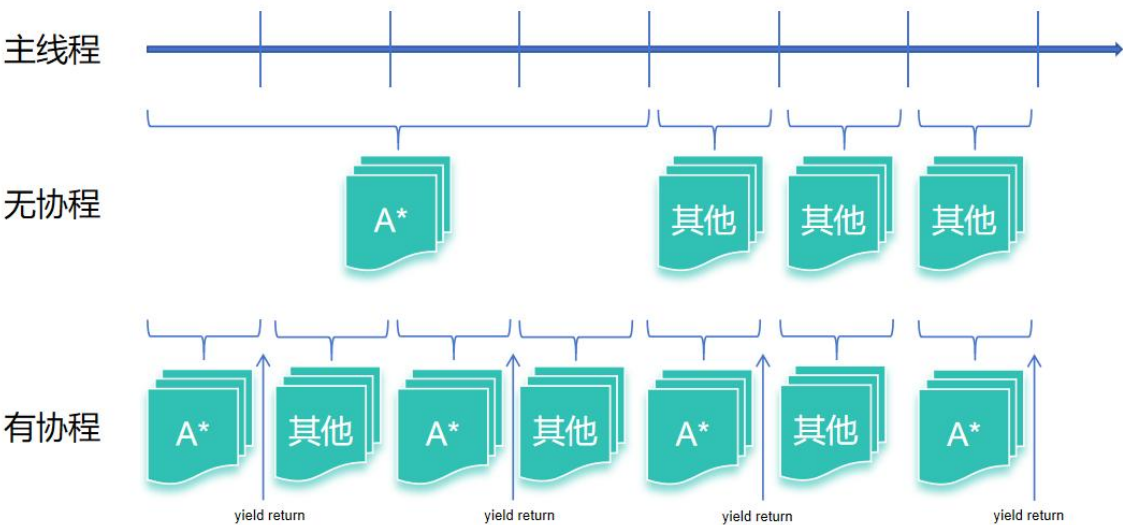


图 3.6 协程在 Unity 中的应用

华为云机器人路径规划模块也有类似的功能，不过不是由协程实现而是基于跳帧。在 A* 路径规划算法运行期间若计算量过大无法在一帧内得到，机器人会维持之前的指令继续运行而不会受到阻塞而停止运动。

在路径规划中，协程可以用 `WaitForSeconds` 或者 `WaitForEndOfFrame` 函数控制执行节奏，当检测到设备性能下降时会减少每帧处理的工作量，相反在高性能硬件上则可以不用。

Unity 引擎内部负责协程的生命周期管理以及执行调度，开发者不用手动处理底层任务队列或者帧同步逻辑，协程的启动凭借 `StartCoroutine` 方法注册到 Unity 主线程的事件循环中，引擎会在每帧更新时自动检查协程的状态，并且根据 `yield return` 指令定义的等待条件比如时间延迟、帧结束、异步操作完成等决定是否继续执行后续代码块。比如当协程中调用 `yield return new WaitForSeconds (2.0f)` 时，Unity 会自动挂起该协程 2 秒后恢复执行，期间不会阻塞主线程的其他操作。

3.4 实时运动

3.4.1 解决“抖动”

角度与速度不匹配以及路径子节点的选择不佳会造成“抖动”式运动，所以本设计针对此方面做出了对比与优化。设计可以在菜单里进入基于华为云的机器人路径规划模块，选择角度与速度不匹配式的“抖动”对比，通过旧版本代码与新版本代码对比运行，可以直观的感受到代码变动产生的影响。以及进入 Unity 机器人路径规划模块，选择路径子节点的选择不佳式的“抖动”对比。在系统中后面会通过白盒测试[4.2.2]来对比没有优化和优化过的实际运行情况，以此来更直观的获得演示效果。

为何会产生角度与速度不匹配式的“抖动”？以下是我的理解与图例。蓝色为机器人，绿色为给出路径，箭头为正面朝向。此时实际的运动过程类似于抖动。究其原因因为机器人在移动过程中与目标间的角度会变化，由于代码里给出了差角过大停止运动，所以每当机器人转角大于 $\pi/6.0$ 时就会减速直至静止，产生如图 3.7 的效果。

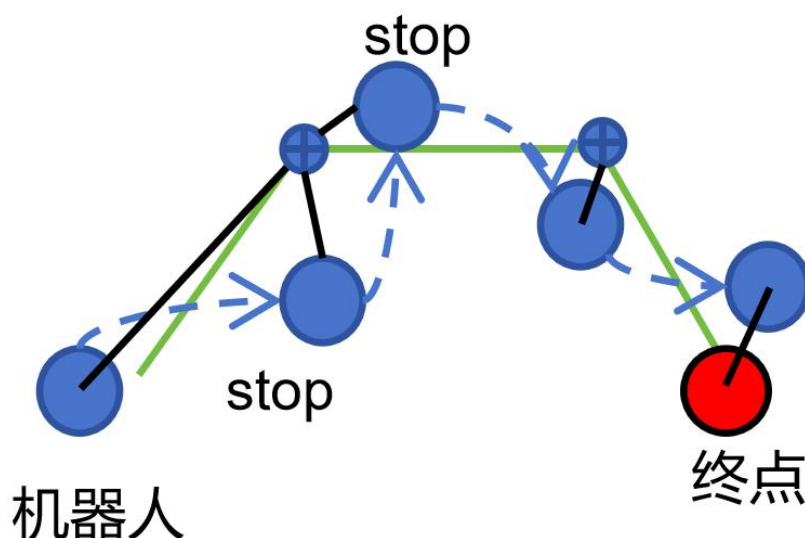


图 3.7 角度与速度不匹配

而为了解决角度与速度不匹配式的抖动，需对应不同角度分别给予不同的速度，在相对角过大时停下中的角度选择也得考虑，实际上就是精度与数据问题。其中伪代码与注释如下：

```

need_dir = radian(rob.x,rob.y, aimx, aimy);           //计算到达目标需要的角度
dir = rob.face_direct;                                //当前角度
d_dir = need_dir - dir;                                //此处为计算需要转向的角度

if (d_dir > PI/6.0 ){                                  //角度大于 PI/6 时速度为 0
    printf("forward %d %f\n", robot_id, 0);}
else if (d_dir < PI/6.0 && d_dir > PI/9.0){           //角度在 PI/6 与 PI/9 之间
    printf("forward %d %f\n", rob_id, max/4.0);}
else if (d_dir <= PI/9.0 && d_dir > PI/12.0){         //以此类推
    printf("forward %d %f\n", rob_id, max/3.0);}
.....                                                //以此类推
    
```

为何会产生路径子节点的选择不佳式的“抖动”？究其原因是使用 A* 路径规划算法时没有采用 8 向搜索，而是采用 4 向搜索导致路径均为直角转角。解决方法：要么采用路径压缩去除角节点的部分子节点使直角路径转变为斜向路径，这种方法低效而且后处理，加大计算量。要么在启发函数的拓展节点时考虑斜向节点，即 8 向搜索可以解决。这种方法简单易实现。具体解析如下图 3.8 所示。

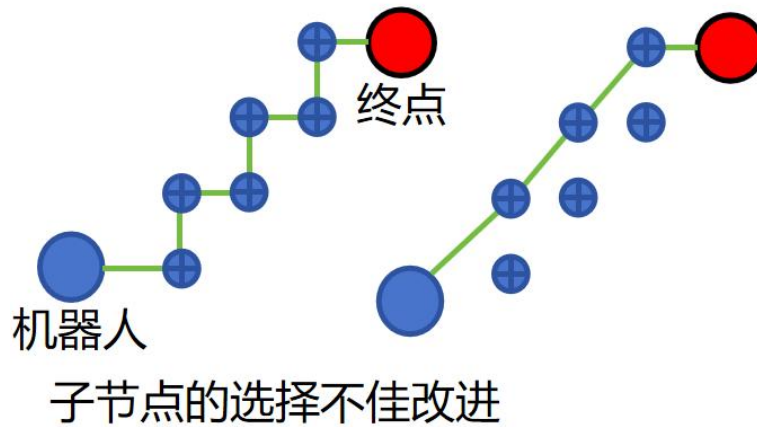


图 3.8 路径子节点的选择不佳

3.4.2 排除“陷阱”

机器人拥有实体，无法通过小于体积的通道处，且在得分系统中由于运送货物会使机器人的体积增大，导致无货物时能通过的路口将不可通行。因此需要对地图检索，标记特殊点位（暂且称为“陷阱”通道，因为通常情况下可以通过，而持有货物不可通过导致无法得分）。而在 A* 路径规划算法中，路径是以二维数组为基础，每一个矩阵节点无详细大小之分，所以如果不加以判断，会导致只要是一个节点大小的通路都会被计算进去，而在运行过程中去除掉此类路径则称之为排除“陷阱”。具体陷阱可分为以下几类，如图 3.9 所示：

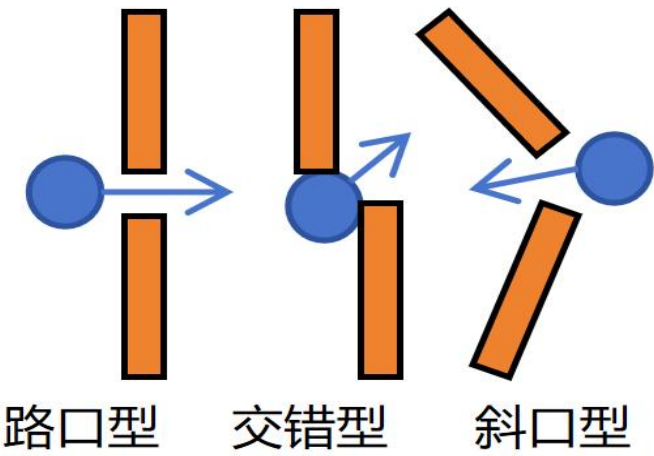


图 3.9 几种常见的“陷阱”

所以在处理“陷阱”上，本设计使用了两种方法：

(1) 扫描地图使用 if() 语句判断“陷阱”。即对于华为云机器人路径规划系统的文本.txt 格式生成的地图而言，先扫描一遍矩阵 `map[101][101]` 的 100×100 的文本地图。然后根据地图的描述字符寻找到 ‘#’ 墙壁字符，根据不同类型的“陷阱”检查周围的矩阵节点是否满足“陷阱”。

(2) 基于 Collider2D 和 Unity 网格去除“陷阱”。Unity 虽然无法用矩阵存储地图信息，但是可以使用 Collider2D 生成“透明”的碰撞框，然后基于碰撞框占用的网格来确定无法通行的区域，之后基于网格运算 A* 路径规划算法。得益于 Unity 主线程运行特性与 Prefab 的应用，可以在运行时生成新的机器人与目标节点，也能实时生成墙体。

3.5 图形用户界面

3.5.1 用户界面结构

GUI 是图形用户界面的首字母缩写，以下简称 UI。在 Unity 平台的支持下，本设计开发了能够实时指定机器人目的地的系统。通过 UI 输入机器人数量、指定机器人更改目的地坐标等等功能，为以后拓展开发额外的模块提供了接口。

UI 系统由 Canvas 画布和 Event System 事件系统组成。能够构建可视化交互界面。其中的 Rect Transform 组件可以自适应 UI 布局，能为不同分辨率屏幕适配大小。开发者可利用组件库快速创建按钮、滑动条、文本框等，结合事件系统实现点击、拖拽等交互响应，这大大的提高了前端界面的开发效率。Unity 内部 UI 的结构如下图 3-10 所示：

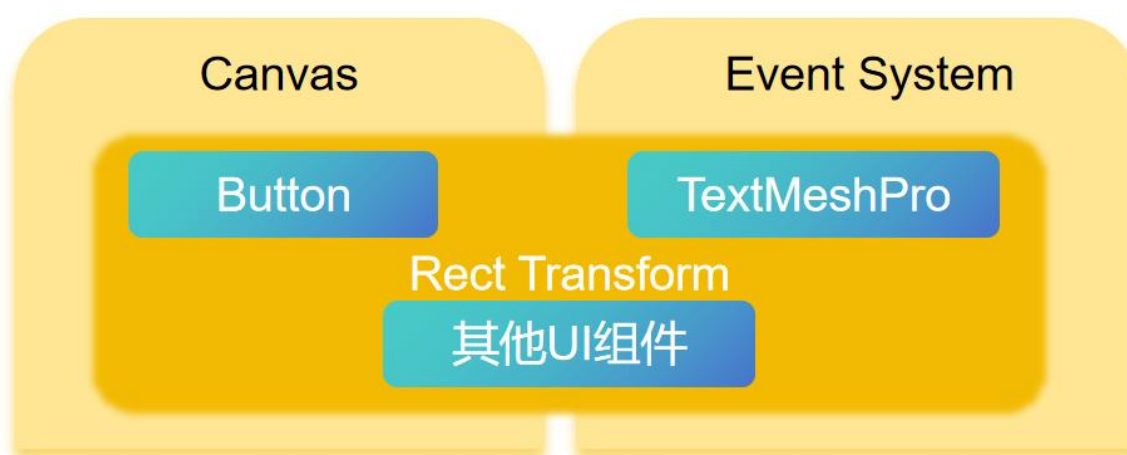


图 3.10 Unity UI 结构

本设计中主要使用到了 Unity 提供的 Button 组件与 Text Mesh Pro 组件搭建 UI 界面。其中按钮的实现只需要创建一个公有函数：

```
public void OnClick(){  
    //执行代码  
}
```

之后在 Button 事件系统内部创建一个 RunTime Only 的运行时才可以触发的事件，将脚本 Uicontrol.cs 的 C#脚本内的 OnClick()函数挂载到事件上即可。之后这个按钮就会在 Unity 主线程中每帧都可以调用，点击按钮就会触发执行代码。

3.5.2 用户界面的单例模式

单例模式(Singleton Pattern)是一类创建型模式，要求在设计某类操作或者对象时保证整个程序在运行期间只存在一个实例对象且易于被访问，即为一种对实例的生成过程进行管理的模式^[10]。单例模式仅仅只提供了一个入口,可以防止同时有多个角色触发这个事件，也可以防止单个角色同时多次触发该事件。在代码编写中如果定义一个非单例模式实例虽可以使得该实例随时被访问，却也可能实例化出多个对象，不能保证实例的唯一性。所以若在一个系统中只允许一个实例存在，那么就应采用单例模式。

所以单例模式类似于一个锁，可以类比于本科《操作系统》课程里学过的进程锁，用于同步与互斥。而这里显然是互斥的，临界区的信号量为 1，即当角色触发该事件后，通过代码给该事件上锁信号量减少。之后除非退出该事件，不然将不能重复触发。

3.5.3 图形用户界面展示

首先打开系统会进入图形用户界面主界面，此界面包含本设计的题目与作者信息，以及三个按钮。通过按钮能够加载两个主要的路径规划模块以及关闭本机器人路径规划系统，如下图 3.11 所示。

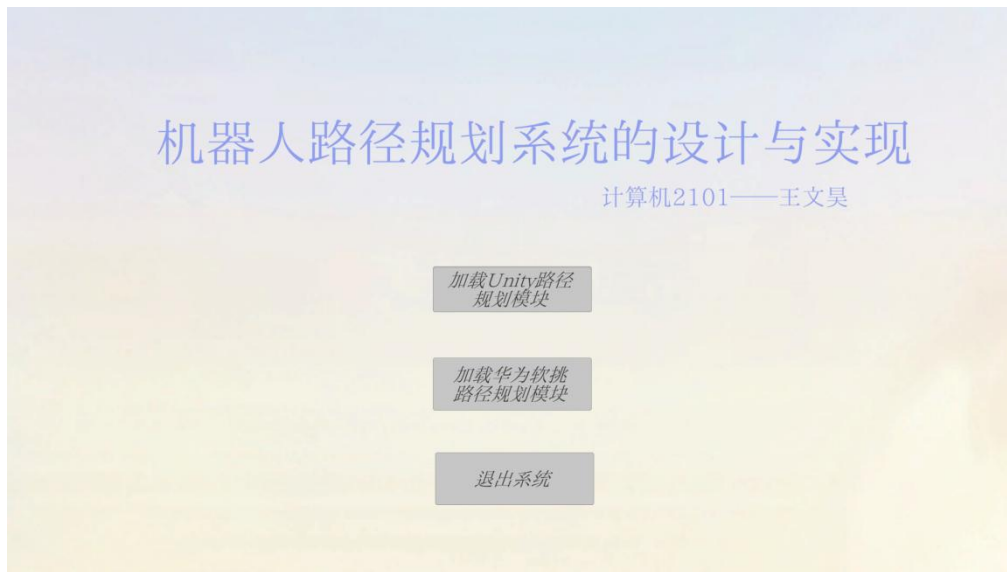


图 3.11 图形用户界面主界面

点击主界面的“加载 Unity 路径规划模块”按钮会进入该模块主界面，此界面包含三个按钮，中间两按钮分别对应此模块的不同功能，能够加载配置路径规划，加载改进运动的对比功能，以及返回前一界面的功能。如图 3.12 所示。

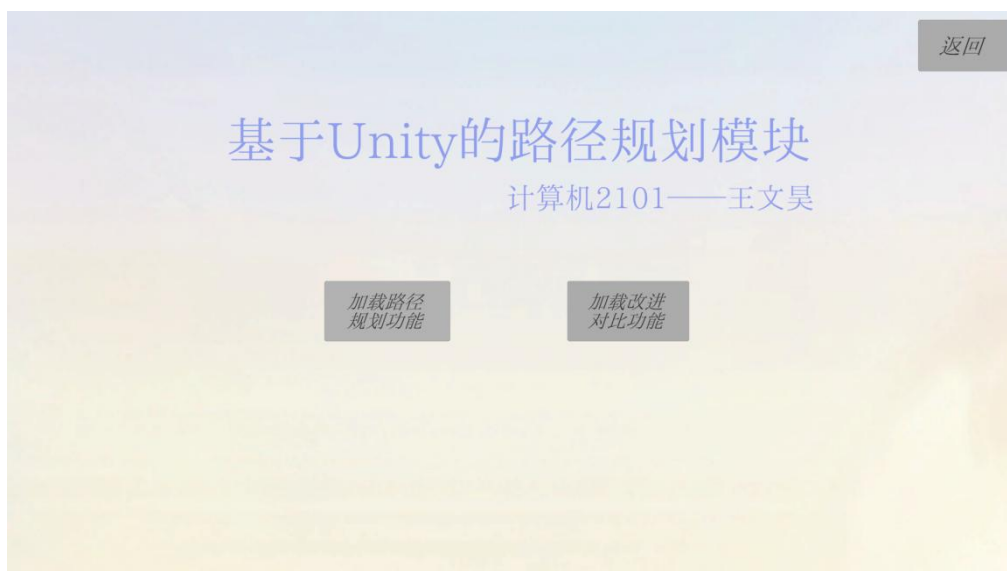


图 3.12 Unity 路径规划模块

点击 Unity 路径规划界面的“加载路径规划功能”按钮后会进入该功能界面，此界面包含五个按钮，三个文本输入框。首先，右上角的返回按钮可以返回上一个界面，左下

角的生成机器人和生成墙体按钮用于生成路径规划所必须的障碍物与机器人和目标节点，而右侧的三个输入框与按钮可以用于指定对应编号的机器人前往对应的坐标位置，编辑完成后点击开始寻路即可执行路径规划功能。初始界面如下图 3.13 所示。

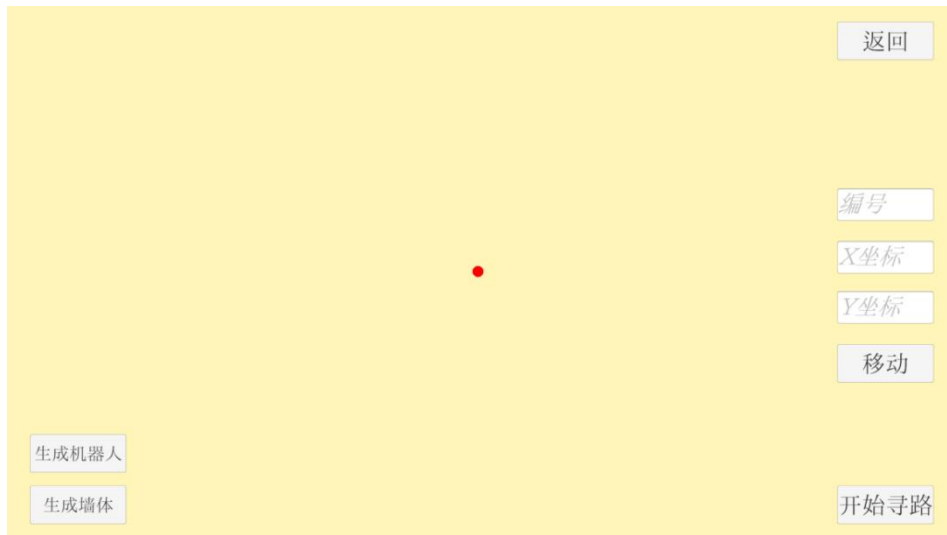


图 3.13 路径规划功能界面

该功能模块基于红点的位置可以按下 WASD 进行上下左右的视角移动，然后在对应位置生成机器人和障碍物，此内容由用户自定义编辑。同时鼠标可以放置在对应的机器人上以查看机器人的序号和具体位置。如下图 3.14 所示

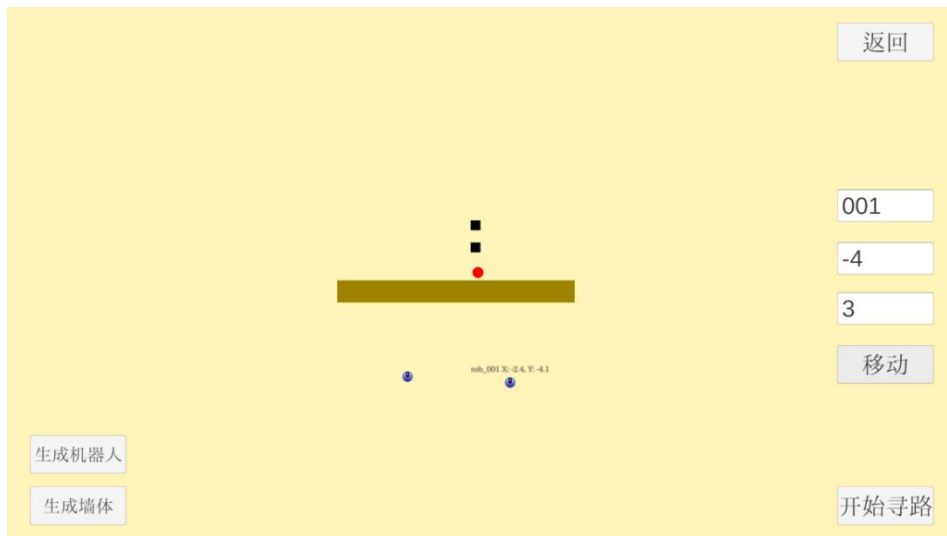


图 3.14 用户自定义创建内容

在用户编辑完成后即可点击按钮“开始寻路”，此时机器人将会主动前往与之对应的目的地。如下图 3.15 所示

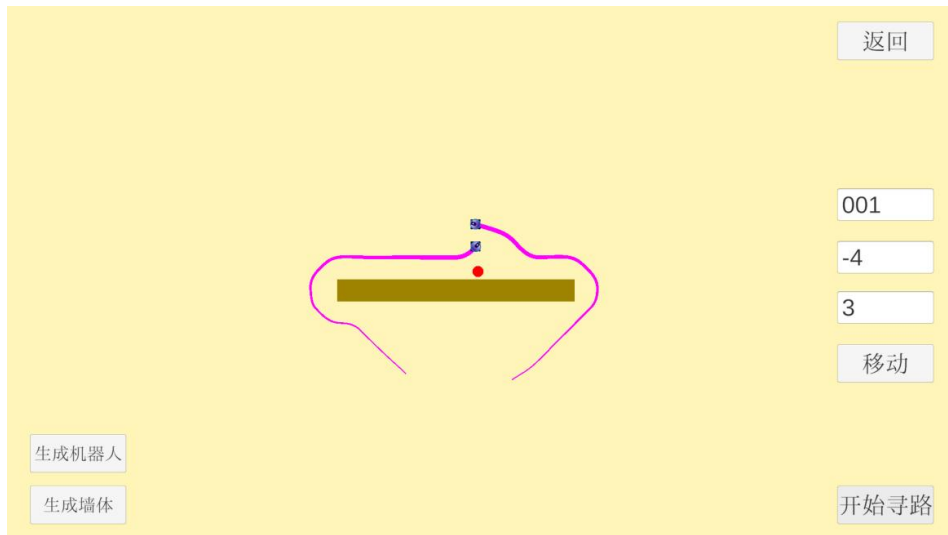


图 3.15 执行路径规划功能

点击 Unity 路径规划界面的“加载华改进对比功能”按钮后会进入该功能主界面，此界面包含三个按钮，点击返回即可返回上一级界面，点击图示中央两个按钮即可加载 A*算法对应的 4 个方向与 8 个方向拓展 OpenList 的路径对比。如下图 3.16 所示。

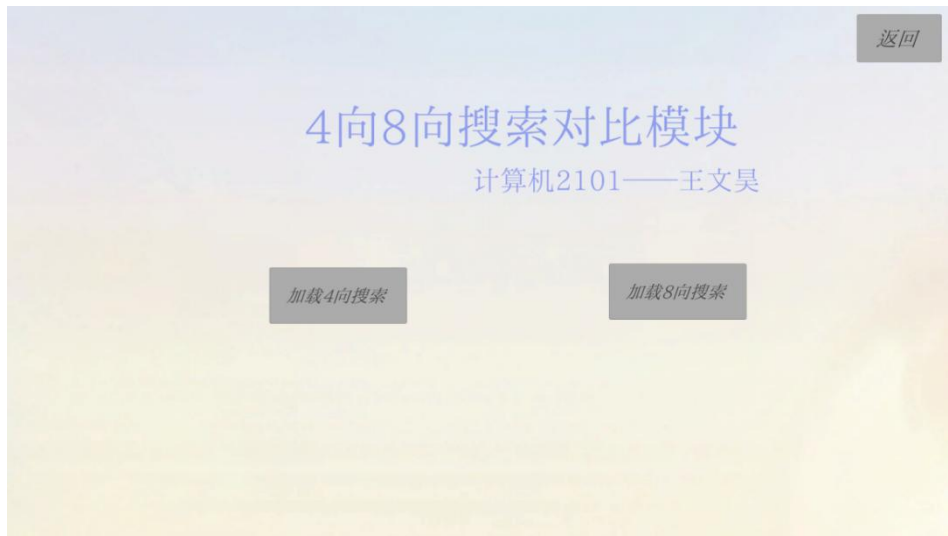


图 3.16 改进对比功能

点击对比功能界面的“加载 4 向搜索”按钮后会进入该功能，此界面仅包含退出按钮。打开该功能将自动演示 4 向拓展的 A* 算法，无需用户自己操作。如下图 3.17 所示。

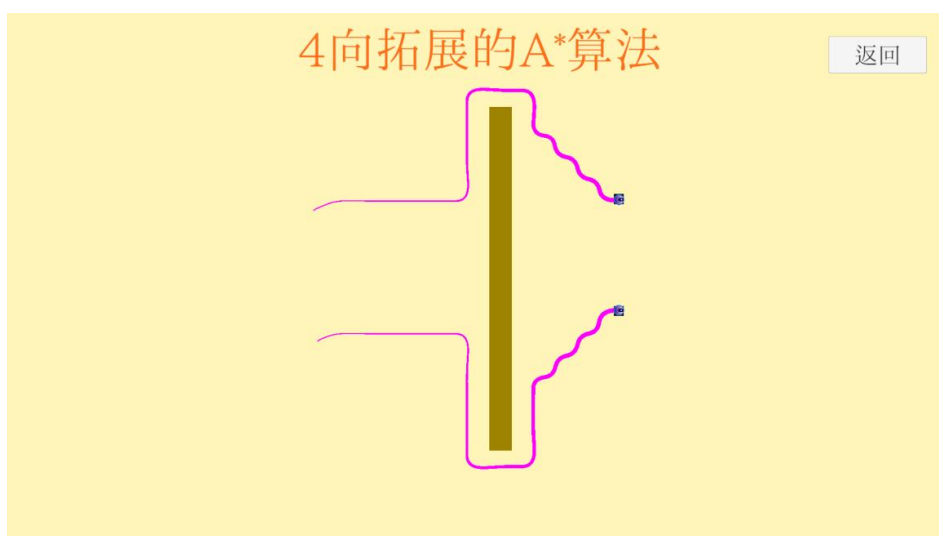


图 3.17 4 向拓展的 A* 算法演示界面

点击主界面的“加载华为软挑路径规划模块”按钮后会进入该模块主界面，此界面包含三个按钮，一个文本输入框。“打开示例”按钮会打开地图，而下方的输入框可以输入并打开自己通过文本设计的地图。如下图 3.18 所示。



图 3.18 华为软挑路径规划模块

点击“打开示例”按钮后会进入该模块界面并显示 2.txt 地图，之后无需用户自己操作将开始路径规划。如下图 3.19 所示。



图 3.19 华为软件精英挑战赛路径规划

第 4 章 系统测试

4.1 测试目的

测试的目的是为了确保机器人路径规划系统满足设计的需求且功能正常，具体目标如下：

- (1) 系统是否满足需求，功能是否齐全。对应设计说明书的第 1 章与第 3 章。是否能为华为云机器人的调度提供路径规划，是否能为自定义位置的寻路机器人提供路径规划。
- (2) 检查系统是否满足最初的设计框架。对应设计说明书的第 2 章。数据层、算法层、交互层架构是否完整，整个系统是否按照三层结构设计？是否缺失结构？
- (3) 系统的运行情况以及安全性与稳定性。对应设计说明书的第 3 章。各个模块是否正常运行，优化是否符合需求。

4.2 测试方案

4.2.1 硬件环境配置

本设计的制作与测试以及其环境均从一台个人笔记本电脑上完成。机器的具体环境及配置的具体信息如表 4.1 所示：

表 4.1 软硬件环境配置

配置项	配置数值
PC	Hasee 神州 战神 TX8-CA7DP
操作系统	Windows 10 家庭中文版
CPU	11 th Gen Intel(R) Core(TM) i7-11700 @ 2.5GHz
GPU	NVIDIA GeForce RTX 3060 Laptop GPU
RAM	16.0 GB
可用显存	6 GB

4.2.2 白盒测试

白盒测试又称结构测试、透明盒测试、逻辑驱动测试或基于代码的测试。白盒测试是一种测试用例设计方法，盒子指的是被测试的软件，白盒指的是盒子是可视的，即清

楚盒子内部的东西以及里面是如何运作的。白盒测试全面了解程序内部逻辑结构、对所有逻辑路径进行测试。

(1) 通过旧版本代码与新版本代码在系统中通过指令打开两个进程对比运行，或者通过 Unity 内两个不同版本的脚本挂载到两个不同的组件上对比执行，检查优化的算法与设计是否实现且是否生效。

(2) 通过文本输入矩阵 `map[101][101]` 的 100×100 的文本地图保存至 `maps` 文件夹，并使用 `bs\5-8\change -m maps\XXXX.txt` 指令与 `robot_gui.exe` 通信检测。

4.2.3 黑盒测试

黑盒测试又叫功能测试、数据驱动测试或基于需求规格说明书的功能测试。该类测试注重于测试软件的功能性需求。黑盒测试就像检查一个盲盒，我们不知道盒子内部有什么物品、怎么制作的，只关心它能不能按照说明书工作。

具体来说，黑盒测试就是模拟真实用户的操作来检查软件功能。本设计的机器人路径规划系统：

(1) 测试 Button 的有效性：点击不同 Button 根据其文本描述查看点击后的功能是否符合要求。

(2) 测试 Text Mesh Pro 的输入有效性：在 Text Mesh Pro 的子组件里面的文本输入框内输入数据，例如：在指定机器人到达指定目的地的功能里，输入对应的 `robotnum`、`positionx`、`positiony` 之后点击移动查看执行中的机器人是否改变路径规划。

4.3 测试结果

测试实时运动的优化：路径规划运行过程中解决子节点选择不佳的“抖动”，8 向与 4 向启发搜索的路径对比结果如下两图所示：

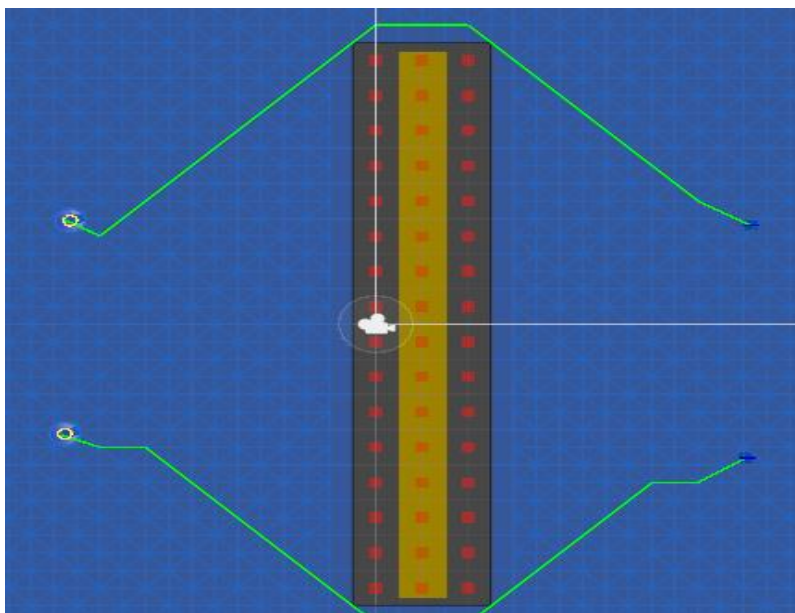


图 4.1 8 向启发搜索

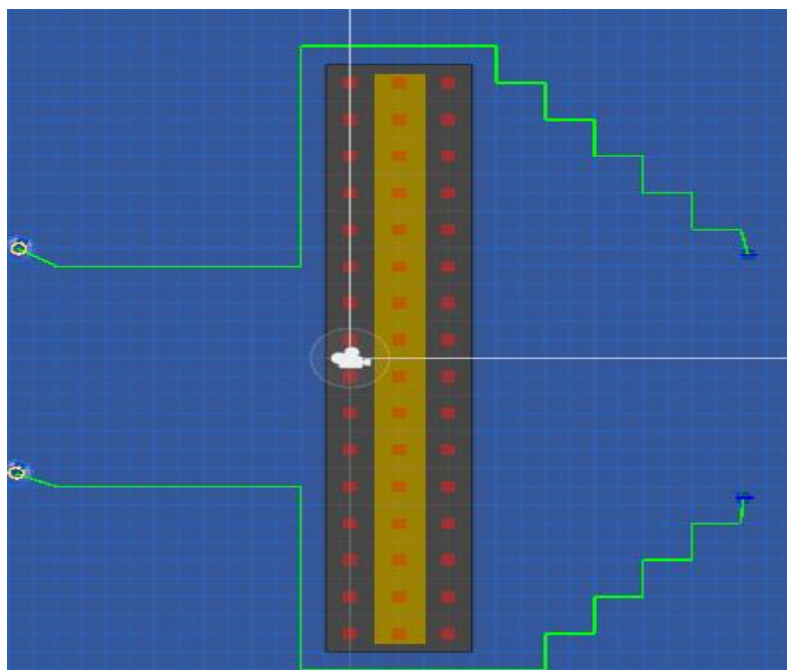


图 4.2 4 向启发搜索

测试地图的生成：根据基于文本的地图模块内存储的 2.txt 文件生成地图。文本如图 4.3 所示：

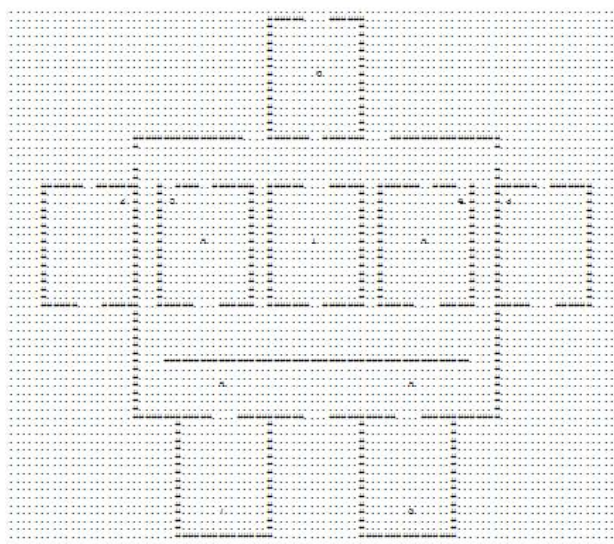


图 4.3 文本地图 2.txt

在系统中通过指令指定文本地图打开基于华为云的路径规划系统，生成的地图如下：

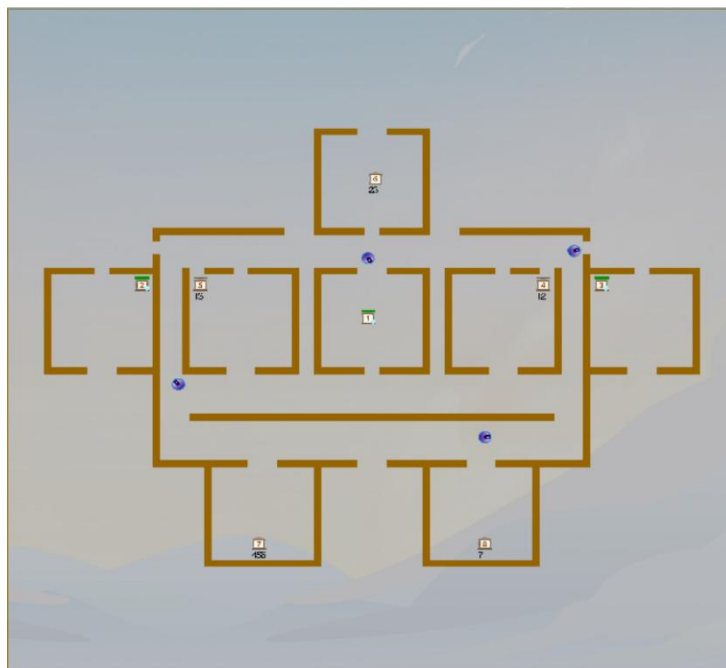


图 4.4 基于文本的地图生成

测试 Button 与 Text Mesh Pro 的有效性: 根据 UI 提示输入对应的机器人序号以及需要前往的 x 坐标与 y 坐标, 如图 4.5 所示。

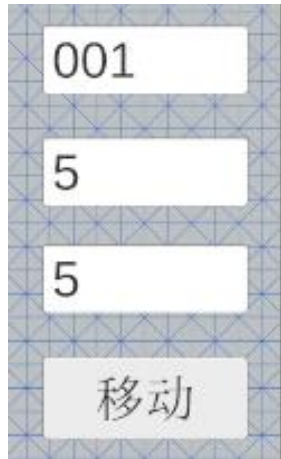


图 4.5 Unity.UI

点击“移动”按钮的同时下方控制台返回对应 Debug 信息, 此时说明 Button 和 Text Mesh Pro 有效。如图 4.6 所示。

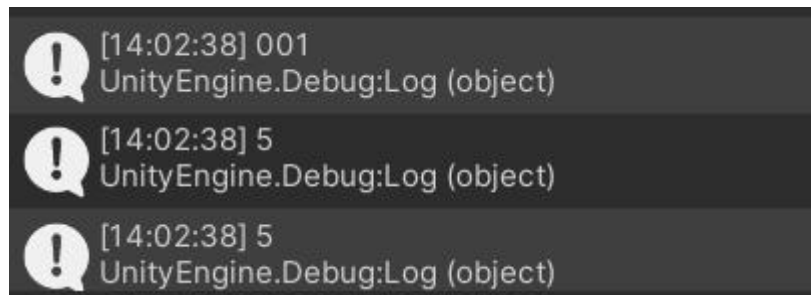


图 4.6 控制台返回 Debug 消息

第 5 章 技术经济可行性分析

5.1 复杂工程问题研究

本机器人路径规划系统满足复杂工程研究问题特性的第一条、第二条、第四条、第七条。

第一条：必须运用深入的工程原理，经过分析才可能得到解决。本设计从系统需求、系统设计与关键技术、系统的实现与优化、系统测试这四个方面深入工程原理，经过分析解决实现过程中遇到的难题，符合第一条特性。

第二条：涉及多方面的技术、工程和其它因素，并可能相互有一定冲突。由于本设计仅仅使用物理引擎进行模拟运动，模拟的运动并非完全等于现实运动，有较大误差。所以该系统在现实世界的实现需要借助电路、机械等行业领域的结合，并可能产生一定的冲突。

第四条：不是仅靠常用方法就可以完全解决的。由于路径规划算法作为 NP-hard，没有最优解，所以不是靠一个固定与常用的方法就能实现的，得结合具体问题具体分析。本设计基于系统需求改进了 A*路径规划算法的启发函数，符合该条特性。

第七条：具有较高的综合性，包含多个相互关联的子问题。本设计除了基于改进 A*路径规划算法用来规划路径以外，还结合了实际的物理引擎实现了实时的物理运动，符合包含多个相互关联的子问题这一特性。同时将该设计的各个模块集成成一个通过可视化图形界面操作的系统，具有较高的综合性。

5.2 技术可行性分析

5.2.1 现有技术支撑

本系统采用的技术基础是 A*路径规划算法，该算法在路径规划领域广泛应用。技术支撑为改进 A*路径规划算法，引入近墙惩罚 $s(n)$ 与经过惩罚 $c(n)$ ，实现了多机器人的路径规划与避障。此改进保留了 A*路径规划算法的核心优势不变，又凭借优化提高了对多机器人多目标适应性。

就开发工具这方面而言，Unity 引擎的平台支持稳定。该公司成立已久，且为个人用户提供免费版本的开源支持。只需通过微信注册个人账号，并按时获取个人许可证即可使用。

5.2.2 跨平台可移植性

Unity 引擎作为十分出名的跨平台开发工具，拥有强大的可移植性。开发者运用 Unity 构建的应用或者系统可以以很低的成本部署到多种终端设备上，实现“一次开发，多端运行”。

不管是桌面端的 Windows、macOS、Linux 系统，还是移动端的 Android、iOS 平台，甚至网页端的 WebGL 或者嵌入式设备，Unity 都提供能无缝集成的打包功能。这种跨平台能力不只是在基础操作系统层面有所体现，还包括对不同硬件架构的适配，开发者不用针对每个平台重新编写核心逻辑，只要凭借 Unity 的统一接口调整一些参数，就能快速适配目标环境，有效缩短开发周期。而且 Unity 的物理引擎和动画系统在不同平台上保持着高度的一致性，让系统在不同环境的实际部署中表现相似，避免了因为平台差异致使调试成本增加。

开发者还可借助 Unity 的 Asset Store 获得跨平台兼容的插件和工具，像输入管理、网络通信模块等，降低多端适配的复杂程度，这种“开箱即用”的特性，让 Unity 成为本项目开发的首选引擎。

5.3 经济可行性分析

5.3.1 引擎的收费模式

Unity 引擎的收费模式历经了多次调整，最终逐渐构形成了一个可兼顾开发者需求和商业可持续性的体系。在早期版本的时候，Unity 引擎以免费使用的方式吸引了众多个人开发者和企业用户，还推出了专业版订阅服务，以此来提供高级功能。

随着生态逐渐成熟，Unity 曾经尝试引入的动态收费机制，也就是要求那些达到一定收入规模的商业项目按照下载次数来支付费用。然而由于政策原因以及技术可行性方面存在争议，遭到了开发者的强烈反对。在此之后，官方又回归到了订阅制的核心框架。并且对免费版与付费版的功能划分进行了优化：个人开发者可免费使用基础功能，只有在项目收入超过特定阈值之后才需要升级至专业版（个人版用户年收入门槛从 10 万美元提升至 20 万美元），企业客户则是依靠定制化订阅来获取专属支持以及高阶工具。

在当前模式下，Unity 借助版本划分来体现服务差异化，比如说免费版保留了跨平台部署以及基础物理引擎，专业版集成了实时协作以及性能分析模块，企业版则提供私有云部署以及专属技术支持。这一模式降低了初创团队的学习成本，还为大型企业提供了深度适配方案。这是当下 Unity 公司在考虑商业化与对开发者友好性之间所进行的平衡探索。

5.3.2 部署代价

本设计基于 Unity 引擎的 2D 功能来开发，2D 项目通常相对 3D 项目有更好的性能与帧率，使得此设计的工程文件打包后可以在一些低端机或者云电脑上运行。若结合跨平台可移植性则能够运行至不同的手机平台，或者移植至网页端。以下表 5.1 是截止至 2025 年 5 月 26 日本设计的各项属性。

表 5.1 设计的属性

属性	源工程文件	打包的可运行文件
空间大小	1.49GB	176MB
Windows 格式	File 源文件	应用程序.exe
运行占用内存	1995.4MB	205.7MB

第 6 章 设计总结

本设计以 2023 年华为软件精英挑战赛赛题为背景，之后在此毕业设计的推动下重新审视了整个程序设计。以系统需求分析为开始，系统性的分析了华为软件精英挑战赛的赛题和机器人寻路系统的需求，将其整合成一个系统的想法就此萌芽。有了想法之后应开始考虑以前用到的技术及之后开发新模块将要要用到的技术，从架构的设计到算法的改进以及平台的选择都经过了深思熟虑，至此想法正式落地。结合 Unity 引擎的开发便利性，我将系统的技术分模块一一实现并能够运行。而后的系统测试与系统可行性分析也是水到渠成。至此本次毕业设计基本结束，但是仍有许多的不足之处。

首先不足的是在 A*路径规划算法的改进上还有许多文章可以做，如是否可以加入更多的参数以达到更多的需求与目的？路径规划的模块是否还有其他的拓展？其次是我认为我的毕业设计的体量不够大、也不够前沿创新，充其量是对于经典路径规划算法的致敬与小小的改进。再者对于界面的设计也不具备十足的艺术感缺乏优质的美术资源，UI 的设计也是一个很好的优化方向，没有做出符合设计科学的图形用户系统界面也是本设计的不足。最重要的一点是由设计最初并没有集成为一个系统，使用十分不方便。好在中期检查的指导老师帮我指出了这一个缺点，让我在后面的开发中有了总体的方向。

算法源自于生活，生活中处处充满算法。在这个信息化的时代，只要你在使用互联网就离不开算法的加持：小到随处可见的短视频推荐，大到人类的太空航行宇宙探索（美国火星探测器核心的寻路算法采用的是 D*（D Star）算法，是 A*算法的变体）。路径规划算法作为 NP-hard 问题对于当前的我来说仍然是一座大山。在此次毕业设计中我根据自己的理解对传统的 A*算法进行了启发函数的改进与实际运动的模拟，但这只能算冰山一角，所以今后我应该更进一步，不断学习与深入了解。

参 考 文 献

- [1] 王建园, 符洋, 陈金宝, 等. 基于改进 A-star 的无人机路径规划算法[J]. 空天技术, 2025, (02):78-87.
- [2] 颜光伟, 胡炜, 尚松超, 等. 一种基于协程的异步网络通信研究[J]. 中国科技信息, 2023, (10):82-85.
- [3] Nobes T K, Harabor D, Wybrow M, et al. The jps pathfinding system in 3d[C]//Proceedings of the International Symposium on Combinatorial Search. 2022, 15(1):145-152.
- [4] 申瑞, 刘佳, 王梦园, 等. 改进 A*算法的移动机器人路径规划设计[J]. 天津职业技术师范大学学报, 2022, 32(01):14-19.
- [5] 沈克宇, 游志宇, 刘永鑫, 等. 基于改进 A*算法的移动机器人路径规划[J]. 计算机应用研究, 2023, 40(01):75-79.
- [6] 欧阳鹏, 孟克. 基于 A*算法的仓库拣货路径规划[J]. 软件, 2024, 45(10):18-20.
- [7] 孙小倩, 辛绍杰. 基于改进型 A*算法的移动机器人路径规划[J]. 组合机床与自动化加工技术, 2023, (03):5-8.
- [8] 何育博, 蔡龙帅, 许乐. 基于改进 A* 算法的移动机器人路径规划[J]. 南方农机, 2024, 55(18):31-34.
- [9] Chen Y, Pang J, Gou Y, et al. Research on the A* Algorithm for Automatic Guided Vehicles in Large-Scale Maps[J]. Applied Sciences, 2024, 14(22):10097-10097.
- [10] 邓伟. Unity 3D 中单例模式的设计与应用[J]. 喀什大学学报, 2020, 41(06):61-64.

致 谢

大学四年的时光匆匆而去，转眼间我将迎来未来的研究生生活。对于本次毕业设计，有太多的感悟与感激无法用言语表达。

本次毕业设计选择机器人路径规划系统的设计与实现，首先要感谢我在 2023 年参与华为软件精英挑战赛的两位队友。因为本毕业设计源于 2023 年华为软件精英挑战赛。感谢参赛队友对我的信任把路径规划的功能交给我开发，使我为本次毕业设计打下了坚实的基础。同时也十分感谢本科毕业设计的指导老师对我的悉心指导！能够让我提前规划与准备好相关事宜，时刻关心我的毕业设计进度。在我的设计上提出了很多很好的建议，帮助我理清思路。也要感谢中期检查的指导老师对我本次设计的认真审查，为我提出了系统集成的建议。最后也感谢学校，为我提供了四年的学习环境。也感谢父母，作为我的坚实后盾守护我的前行。

在今后的研究生学习生涯中，我也会继续学习如何阅读文献与写作。在此再次衷心感谢所有帮助过我的朋友、老师与单位！

附录一 外文原文

附录二 中文译文

附录三 设计代码